Note de séance SQL n°11

Analyses métier : évolution, entonnoir, RFM, cohortes | 2026-11-02

1. Les notions

Analyse	Principe et outils
Évolution mensuelle	Série par mois (DATE_FORMAT, GROUP BY), puis LAG (variation en %), SUM OVER (cumul), SUM OVER (
Entonnoir	Chaque étape est incluse dans la précédente : compter des clients distincts. Deux taux : par rapport à l'étape
RFM	Récence (jours depuis le dernier achat, date de référence fixe), Fréquence (nombre d'achats), Montant. Segm
Cohortes	Cohorte = mois d'inscription. Décalage = PERIOD_DIFF(AAAAMM achat, AAAAMM inscription). Taille de la c
Rapport	Toujours préciser la taille de l'échantillon : un client pèse 20 à 33 points de pourcentage dans une cohorte de

2. Partie 1 : évolution mensuelle et entonnoir (tables commandes, clients, livraisons)

1 : Tableau mensuel

```
SELECT DATE_FORMAT(date_commande, '%Y-%m') AS mois,
       COUNT(*) AS nb_commandes, SUM(montant) AS ca,
       ROUND(AVG(montant), 2) AS panier_moyen
FROM commandes
GROUP BY mois
ORDER BY mois;
```

2026-01 : 2, 290, 145,00 ; 02 : 2, 815, 407,50 ; 03 : 2, 330, 165,00 ; 04 : 1, 120, 120,00.

2-5 : Variation, cumul, part, moyenne mobile, rang

```
WITH ca_mois AS (
    SELECT DATE_FORMAT(date_commande, '%Y-%m') AS mois, SUM(montant) AS ca
    FROM commandes
    GROUP BY mois
)
SELECT mois, ca,
       ROUND((ca - LAG(ca) OVER (ORDER BY mois)) * 100.0
             / LAG(ca) OVER (ORDER BY mois), 1) AS variation_pct,
       SUM(ca) OVER (ORDER BY mois) AS cumul,
       ROUND(ca * 100.0 / SUM(ca) OVER (), 2) AS part_pct,
       ROUND(AVG(ca) OVER (ORDER BY mois ROWS BETWEEN 1 PRECEDING AND CURRENT ROW), 2)
           AS moyenne_mobile,
       RANK() OVER (ORDER BY ca DESC) AS rang
FROM ca_mois
ORDER BY mois;
```

Variation : NULL, 181,0, -59,5, -63,6. Cumul : 290, 1105, 1435, 1555. Parts : 18,65, 52,41, 21,22, 7,72. Moyenne mobile : 290, 552,5, 572,5, 225. Rangs : 3, 1, 2, 4.

6 : Entonnoir en quatre étapes

```
WITH etapes AS (
    SELECT 1 AS ordre, 'Inscrits' AS etape, COUNT(*) AS nb FROM clients
    UNION ALL
    SELECT 2, 'Ont commandé', COUNT(DISTINCT c.id)
    FROM commandes o INNER JOIN clients c ON o.client_id = c.id
    UNION ALL
    SELECT 3, 'Ont une livraison', COUNT(DISTINCT c.id)
    FROM livraisons l
    INNER JOIN commandes o ON o.id = l.commande_id
    INNER JOIN clients c ON c.id = o.client_id
```

```
    UNION ALL
    SELECT 4, 'Livrés', COUNT(DISTINCT c.id)
    FROM livraisons l
    INNER JOIN commandes o ON o.id = l.commande_id
    INNER JOIN clients c ON c.id = o.client_id
    WHERE l.statut = 'livrée'
)
SELECT etape, nb,
       ROUND(nb * 100.0 / LAG(nb) OVER (ORDER BY ordre), 1) AS taux_vs_precedent,
       ROUND(nb * 100.0 / FIRST_VALUE(nb) OVER (ORDER BY ordre), 1) AS taux_vs_depart
FROM etapes
ORDER BY ordre;
```

Inscrits 8 ; Ont commandé 4 (50,0 / 50,0) ; Ont une livraison 3 (75,0 / 37,5) ; Livrés 2 (66,7 / 25,0). La plus forte perte : inscrits vers commandes (4 clients, 50 %). Action : relancer les inscrits sans commande (offre de première commande), simplifier le parcours, interroger ces clients.

3. Partie 2 : segmentation RFM (tables clients_ana et achats)

```
CREATE TABLE clients_ana (id INT PRIMARY KEY, prenom VARCHAR(50) NOT NULL,
                          date_inscription DATE NOT NULL);
CREATE TABLE achats (id INT AUTO_INCREMENT PRIMARY KEY, client_id INT NOT NULL,
                     date_achat DATE NOT NULL, montant DECIMAL(10,2) NOT NULL,
                     CONSTRAINT fk_achats_client FOREIGN KEY (client_id)
                         REFERENCES clients_ana(id));
-- 12 clients (inscrits en janvier, février et mars 2026) et 30 achats,
-- 3825,00 au total (script complet fourni pendant la séance)
```

1-3 : Tableau RFM et segments (CTE réutilisable)

```
WITH rfm AS (
    SELECT c.id, c.prenom,
           DATEDIFF('2026-07-01', MAX(a.date_achat)) AS recence,
           COUNT(*) AS frequence,
           SUM(a.montant) AS montant
    FROM clients_ana c
    INNER JOIN achats a ON a.client_id = c.id
    GROUP BY c.id, c.prenom
),
segments AS (
    SELECT *,
           CASE WHEN recence <= 45 AND frequence >= 3 THEN 'Champion'
                WHEN frequence >= 3 THEN 'Fidèle'
                WHEN recence > 90 THEN 'À relancer'
                ELSE 'Occasionnel' END AS segment
    FROM rfm
)
SELECT prenom, recence, frequence, montant, segment
FROM segments
ORDER BY id;
```

Awa 42, 4, 550 Champion ; Jean 82, 2, 550 Occasionnel ; Marie 11, 6, 345 Champion ; Patrick 152, 1, 500 À relancer ; Grace 121, 2, 200 À relancer ; David 74, 3, 255 Fidèle ; Sarah 137, 1, 200 À relancer ; Eric 87, 2, 100 Occasionnel ; Nadine 27, 4, 580 Champion ; Paul 118, 1, 320 À relancer ; Rachel 46, 3, 135 Fidèle ; Kevin 91, 1, 90 À relancer.

4 : Synthèse par segment

```
WITH rfm AS (
    SELECT c.id, c.prenom,
           DATEDIFF('2026-07-01', MAX(a.date_achat)) AS recence,
           COUNT(*) AS frequence,
           SUM(a.montant) AS montant
    FROM clients_ana c
    INNER JOIN achats a ON a.client_id = c.id
    GROUP BY c.id, c.prenom
```

```
),
segments AS (
    SELECT *,
           CASE WHEN recence <= 45 AND frequence >= 3 THEN 'Champion'
                WHEN frequence >= 3 THEN 'Fidèle'
                WHEN recence > 90 THEN 'À relancer'
                ELSE 'Occasionnel' END AS segment
    FROM rfm
)
SELECT segment, COUNT(*) AS nb_clients, SUM(montant) AS ca,
       ROUND(SUM(montant) * 100.0 / SUM(SUM(montant)) OVER (), 2) AS part_ca
FROM segments
GROUP BY segment
ORDER BY ca DESC;
```

Champion 3, 1475, 38,56 % ; À relancer 5, 1310, 34,25 % ; Occasionnel 2, 650, 16,99 % ; Fidèle 2, 390, 10,20 %.

5 : Quartiles de dépense

```
SELECT c.prenom, SUM(a.montant) AS montant,
       NTILE(4) OVER (ORDER BY SUM(a.montant)) AS quartile
FROM clients_ana c
INNER JOIN achats a ON a.client_id = c.id
GROUP BY c.id, c.prenom
ORDER BY montant;
```

Quartile 1 : Kevin, Eric, Rachel ; 2 : Grace, Sarah, David ; 3 : Paul, Marie, Patrick ; 4 : Awa, Jean, Nadine.

6-7 : Écart moyen entre achats (base des seuils)

```
SELECT ROUND(AVG(ecart), 1) AS ecart_moyen_jours
FROM (
    SELECT DATEDIFF(date_achat,
                    LAG(date_achat) OVER (PARTITION BY client_id ORDER BY date_achat)) AS ecart
    FROM achats
) t
WHERE ecart IS NOT NULL;
```

37,0 jours : un silence de 90 jours vaut plus de deux fois et demie le rythme habituel. Patrick (un achat de 500, 152 jours de silence) est À relancer : le montant seul masque le risque.

4. Partie 3 : cohortes

1 : Taille des cohortes (depuis clients_ana seul)

```
SELECT DATE_FORMAT(date_inscription, '%Y-%m') AS cohorte, COUNT(*) AS nb_clients
FROM clients_ana
GROUP BY cohorte
ORDER BY cohorte;
```

2026-01 : 5 ; 2026-02 : 4 ; 2026-03 : 3. La taille ne se calcule pas avec la jointure sur achats : on perdrait les clients qui n'ont jamais acheté.

2-3 : Activité par décalage et rétention

```
WITH tailles AS (
    SELECT DATE_FORMAT(date_inscription, '%Y-%m') AS cohorte, COUNT(*) AS nb_clients
    FROM clients_ana
    GROUP BY cohorte
),
actifs AS (
    SELECT DATE_FORMAT(c.date_inscription, '%Y-%m') AS cohorte,
           PERIOD_DIFF(DATE_FORMAT(a.date_achat, '%Y%m'),
                       DATE_FORMAT(c.date_inscription, '%Y%m')) AS decalage,
           COUNT(DISTINCT c.id) AS nb_actifs
    FROM clients_ana c
    INNER JOIN achats a ON a.client_id = c.id
    GROUP BY cohorte, decalage
```

```
)
SELECT a.cohorte, a.decalage, a.nb_actifs, t.nb_clients,
       ROUND(a.nb_actifs * 100.0 / t.nb_clients, 2) AS taux_activite
FROM actifs a
INNER JOIN tailles t ON t.cohorte = a.cohorte
ORDER BY a.cohorte, a.decalage;
```

Janvier : 80,00, 60,00, 60,00, 40,00, 40,00, 20,00 ; février : 75,00, 50,00, 75,00, 25,00, 25,00 ; mars : 66,67, 66,67, 33,33.

4 et 6 : Matrice de rétention, avec cohortes non observables à NULL

```
WITH tailles AS (
    SELECT DATE_FORMAT(date_inscription, '%Y-%m') AS cohorte,
           PERIOD_DIFF(202606, DATE_FORMAT(MIN(date_inscription), '%Y%m')) AS mois_obs,
           COUNT(*) AS nb_clients
    FROM clients_ana
    GROUP BY cohorte
),
actifs AS (
    SELECT DATE_FORMAT(c.date_inscription, '%Y-%m') AS cohorte,
           PERIOD_DIFF(DATE_FORMAT(a.date_achat, '%Y%m'),
                       DATE_FORMAT(c.date_inscription, '%Y%m')) AS decalage,
           COUNT(DISTINCT c.id) AS nb_actifs
    FROM clients_ana c
    INNER JOIN achats a ON a.client_id = c.id
    GROUP BY cohorte, decalage
)
SELECT t.cohorte, t.nb_clients,
  CASE WHEN 0 <= t.mois_obs THEN ROUND(COALESCE(SUM(CASE WHEN a.decalage = 0
       THEN a.nb_actifs END), 0) * 100.0 / t.nb_clients, 1) END AS m0,
  CASE WHEN 1 <= t.mois_obs THEN ROUND(COALESCE(SUM(CASE WHEN a.decalage = 1
       THEN a.nb_actifs END), 0) * 100.0 / t.nb_clients, 1) END AS m1,
  CASE WHEN 2 <= t.mois_obs THEN ROUND(COALESCE(SUM(CASE WHEN a.decalage = 2
       THEN a.nb_actifs END), 0) * 100.0 / t.nb_clients, 1) END AS m2,
  CASE WHEN 3 <= t.mois_obs THEN ROUND(COALESCE(SUM(CASE WHEN a.decalage = 3
       THEN a.nb_actifs END), 0) * 100.0 / t.nb_clients, 1) END AS m3,
  CASE WHEN 4 <= t.mois_obs THEN ROUND(COALESCE(SUM(CASE WHEN a.decalage = 4
       THEN a.nb_actifs END), 0) * 100.0 / t.nb_clients, 1) END AS m4,
  CASE WHEN 5 <= t.mois_obs THEN ROUND(COALESCE(SUM(CASE WHEN a.decalage = 5
       THEN a.nb_actifs END), 0) * 100.0 / t.nb_clients, 1) END AS m5
FROM tailles t
LEFT JOIN actifs a ON a.cohorte = t.cohorte
GROUP BY t.cohorte, t.nb_clients, t.mois_obs
ORDER BY t.cohorte;
```

Janvier : 80,0, 60,0, 60,0, 40,0, 40,0, 20,0 ; février : 75,0, 50,0, 75,0, 25,0, 25,0, NULL ; mars : 66,7, 66,7, 33,3, 0,0, NULL, NULL. Un mois observé sans achat vaut 0,0 ; un mois qui n'a pas encore eu lieu vaut NULL.

5 : Chiffre d'affaires par cohorte

```
SELECT DATE_FORMAT(c.date_inscription, '%Y-%m') AS cohorte,
       COUNT(DISTINCT c.id) AS nb_clients, SUM(a.montant) AS ca,
       ROUND(SUM(a.montant) / COUNT(DISTINCT c.id), 2) AS ca_par_client
FROM clients_ana c
INNER JOIN achats a ON a.client_id = c.id
GROUP BY cohorte
ORDER BY cohorte;
```

2026-01 : 5, 2145,00, 429,00 ; 2026-02 : 4, 1135,00, 283,75 ; 2026-03 : 3, 545,00, 181,67.

5. Lecture de la matrice et recommandation

• **Constat** : la rétention baisse après le mois d'inscription ; M0 n'est pas à 100 % (le client s'inscrit sans acheter tout de suite : 80 %, 75 % et 66,7 %). Un noyau de clients fidèles subsiste (20 % de la

cohorte de janvier achète encore au mois 5).

• **Interprétation** : beaucoup de clients n'achètent qu'une fois. Le rythme habituel est d'environ 37 jours entre deux achats.

• **Action** : relancer chaque client environ 30 jours après son premier achat (rappel ou offre de deuxième achat), et suivre le taux de retour au mois 1 comme indicateur.

• **Précaution** : avec 3 à 5 clients par cohorte, ces taux sont fragiles ; à confirmer sur un échantillon plus grand.

6. Pièges rencontrés

• **Taille d'une cohorte** : à calculer sur la table des clients, pas après une jointure sur les achats (on perdrait les clients sans achat).

• **NULL ou 0** : un mois observé sans achat vaut 0 ; un mois qui n'a pas eu lieu vaut NULL (cohortes non observables).

• **Entonnoir sans taux** : les effectifs seuls ne montrent pas où l'on perd le plus ; calculer les taux avec LAG et FIRST_VALUE.

• **Tri** : un ORDER BY dans un CTE n'est pas conservé ; trier dans la requête finale (ex. ORDER BY ca DESC).

• **Noms** : écrire les segments et les étiquettes exactement (« À relancer »), sans accent dans les alias, avec une casse cohérente pour les CTE.

• **Date de référence fixe** : avec CURDATE(), la récence change chaque jour et les segments aussi ; une date fixe rend l'analyse reproductible.

• **Recommander** : une analyse sans action proposée n'est pas terminée. Les seuils se discutent avec le métier, en s'appuyant sur les données.